

AhamX Bodhi

Where I Becomes Infinite

Autoregressive Generation

LLM Fundamentals — Module 1.2
Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- How LLMs generate text one token at a time
- The autoregressive prediction loop
- Left-to-right conditional probability
- How past tokens shape future predictions

Foundation Concepts

- Connects to: Probability and Token Prediction
- Leads to: Model Sizing and Scaling Laws
- Core mechanism behind GPT, Llama, Mistral
- Duration: approximately 8 minutes



Learning Objectives

By the End of This Concept You Will Be Able To

1. **Recall** what autoregressive generation means in language models
2. **Explain** the step-by-step token prediction process
3. **Describe** the role of conditional probability in generation
4. **Apply** the concept to understand how GPT-style models produce text
5. **Connect** autoregression to real-world LLM behaviour



Prerequisites and Context

You Should Already Know

- What a language model is
- Probability and token prediction
- Tokens as the basic unit of text

What This Unlocks

- Understanding model output generation
- Basis for sampling strategies
- Foundation for fine-tuning objectives
- Bridge to Transformer architecture



What is Autoregressive Generation?

Core Definition

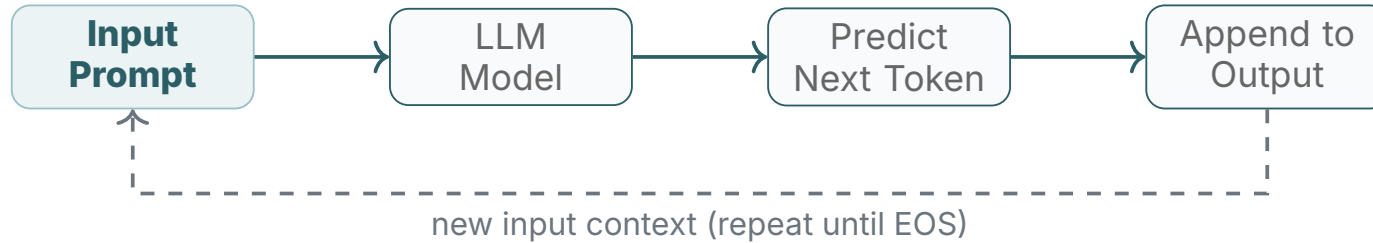
Autoregressive generation is a process in which a model produces output *one token at a time*, where each new token is predicted based on *all previously generated tokens*.

- "Auto" means self-referential: the model uses its own prior outputs
- "Regressive" means it conditions on past values to predict the next
- The model *consumes its own output* as new input at every step

When you send a prompt to an LLM, the reply is assembled token by token through this mechanism — no word is chosen without considering everything written before it.



The Autoregressive Loop --- Step by Step



Loop Steps

- **Step 1** — Feed prompt into model
- **Step 2** — Model predicts the most likely next token
- **Step 3** — Append token; use full sequence as next input
- **Step 4** — Repeat until End-Of-Sequence (EOS) token



The Mathematics of Autoregression

Conditional Probability Factorisation

Given a sequence of tokens $x_1, x_2, \dots, x_T$, the joint probability factorises as:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

- Each factor $P(x_t \mid x_{<t})$ is one forward pass of the LLM
- The model outputs a probability distribution over the entire vocabulary
- A **sampling strategy** (greedy, temperature, Top- p) selects the actual token from that distribution

Training minimises the cross-entropy loss between the predicted $P(x_t \mid x_{<t})$ and the true next token in the corpus — making this probability chain the heart of both training *and* inference.



Token-by-Token Example

Prompt: "The cat sat on"

Watch how each generation step extends the sequence:

Step	Context (Input)	Top Candidates	Chosen
1	"The cat sat on"	the / a / my / ...	"the"
2	"The cat sat on the"	mat / floor / roof / ...	"mat"
3	"The cat sat on the mat"	. / and / , / ...	."
4	"The cat sat on the mat."	EOS / The / ...	EOS

Key Observations

- Longer context leads to a more constrained next-token distribution
- The model retains all prior tokens via the attention mechanism
- EOS token signals the model to stop generating



Left-to-Right Generation --- Causal Language Models

Why Left-to-Right?

- Future tokens are unknown at prediction time
- Causal masking enforces this constraint
- Token x_t can only attend to $x_1 \dots x_{t-1}$
- Prevents look-ahead during training

Decoder-Only Models

- GPT, GPT-2, GPT-4 are decoder-only
- Llama, Mistral, Falcon — all decoder-only
- Masked self-attention enforces causal order
- Contrast: BERT is bidirectional, not generative



Sampling Strategies --- Choosing the Next Token

Deterministic Strategies

- **Greedy** — always pick highest-probability token
- **Beam search** — keep top- k candidate sequences
- Consistent, often repetitive output
- Good for structured tasks (code, translation)

Stochastic Strategies

- **Temperature** — scale logits before softmax
- **Top- p (Nucleus)** — sample from cumulative mass
- **Top- k** — restrict to top k candidates
- Adds creativity and diversity to output



Temperature --- Controlling Randomness

Temperature Formula

Before sampling, logits z_i are divided by temperature τ :

$$P(x_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

- $\tau \rightarrow 0$: distribution sharpens — nearly greedy (deterministic)
- $\tau = 1$: unchanged model distribution (default)
- $\tau > 1$: distribution flattens — more random, creative output
- Typical range in production: 0.2 to 1.2

The temperature parameter in the Claude and OpenAI APIs implements this exact formula at inference time.



GenAI Application --- Conversational AI

How Chatbots Use Autoregression

- The entire conversation history is concatenated into one long prompt
- Every user turn and assistant turn becomes part of the context window
- The model predicts the next assistant token given all prior dialogue
- Multi-turn memory is achieved purely through context, not persistent state

Production Context Windows

- GPT-4 Turbo: 128K tokens — supports very long conversations
- Claude 3.5: 200K tokens — entire books can serve as context
- Longer context means more autoregressive steps and higher compute cost



GenAI Application --- Code Generation

Autoregression in Code Assistants

- GitHub Copilot, Cursor, CodeWhisperer are all autoregressive
- File content and cursor position form the prompt; completion is the prediction
- Each line of code is generated token by token, including indentation
- Fill-in-the-Middle (FIM) extends basic left-to-right for code infilling

Key Insight

- Structural constraints such as valid syntax emerge from next-token statistics
- The model has seen billions of code tokens during pre-training
- Context before the cursor is the autoregressive conditioning signal



GenAI Application --- Summarisation and Document QA

Document Summarisation Pipeline

- Source document plus instruction forms the prompt fed to the LLM
- Model autoregressively generates a condensed version
- Each summary token is conditioned on the full document context

RAG Integration

- Retrieved passages are prepended to the prompt
- The autoregressive answer is grounded in the retrieved context
- Hallucination risk reduces when retrieved context is strong
- Used in legal document review and medical literature summarisation



GenAI Application --- Creative Writing and API Usage

Creative Generation Properties

- Story continuation, poetry, screenplay — all autoregressive
- Higher temperature produces surprising word choices and more creative output
- The model maintains narrative coherence via full context conditioning
- Genre, style, and tone are implicitly encoded in the prompt

API Parameters That Control Autoregression

- `temperature`, `top_p`, `top_k`, `max_tokens`
- `stop` sequences allow early termination of generation



Autoregression vs. Alternative Generation Paradigms

Property	Autoregressive	Masked LM	Diffusion (text)
Direction	Left-to-right	Bidirectional	Iterative denoising
Training target	Next-token pred.	Masked token pred.	Noise reversal
Best for	Generation	Classification	Structured output
Inference speed	Sequential	Parallel	Iterative
Examples	GPT-4, Llama	BERT, RoBERTa	MDLM, PLAID
Context used	Causal only	Full context	Full context

AR models dominate general-purpose LLM applications (2024–2025) due to flexibility in open-ended generation.



Limitations and Challenges

Known Limitations

- **Sequential bottleneck** — each token requires prior ones
- **Error propagation** — wrong token corrupts future output
- **Context window** limits how far back the model can see
- **Slow inference** for long outputs (no generation parallelism)

Active Research Mitigations

- **Speculative decoding** — draft and verify tokens in parallel
- **KV-cache** — reuse attention computations across steps
- **Flash Attention** — memory-efficient attention
- **Longer context** models reduce truncation errors

Skills You Are Building



Conceptual Skills

- Probabilistic thinking about language
- Understanding generation vs. classification
- Connecting training objectives to inference
- Reasoning about context and memory in LLMs

Applied Skills

- Interpreting API parameters such as temperature and top-p
- Designing prompts that exploit autoregressive conditioning
- Debugging hallucinations by examining context quality
- Choosing the right model for generation vs. encoding tasks



Professional Role Relevance

Role	How Autoregression Knowledge Applies
ML Engineer	Optimise inference pipelines; tune sampling parameters
AI Engineer	Build chatbot and RAG applications on top of LLM APIs
Data Scientist	Interpret model outputs; understand generation diversity
Product Manager	Set expectations around generation latency and quality
Research Scientist	Improve decoding strategies; design new generation objectives
Software Developer	Integrate LLM APIs with correct parameter configuration



Where This Sits in Your Learning Journey

Prerequisites

- What is a Language Model?
- Probability and Token Prediction
- Tokens as discrete vocabulary units

This Concept Enables

- Model Sizing and Scaling Laws
- Foundation Models Overview
- API Parameters (Temperature, Top- p)
- Transformer Architecture (Module 2)
- Fine-tuning objectives (Module 7)



Related Concepts in This Programme

Connections Across the Curriculum

- **Causal Language Modelling (M9)** — pre-training objective for AR models
- **Decoder-Only Architecture (M3)** — the Transformer design enabling AR generation
- **Temperature and Top- p Parameters (M3)** — sampling from the AR distribution
- **RLHF (M7)** — reward model guides which AR outputs are preferred
- **RAG (M5)** — retrieved documents extend the autoregressive context
- **Hallucination and Grounding (M8)** — AR errors and mitigation strategies



Common Misconceptions

Do Not Confuse These

- **"The model plans the whole sentence before writing it"** — *False*. Each token is sampled from $P(x_t \mid x_{<t})$; there is no look-ahead.
- **"Higher temperature means smarter output"** — *False*. Higher temperature increases diversity, not reasoning ability.
- **"The model looks at future tokens during generation"** — *False*. Causal masking strictly prevents this.
- **"BERT generates text autoregressively"** — *False*. BERT is a masked LM; it is not a generative model.



Summary --- Autoregressive Generation

Key Takeaways

1. LLMs generate text **one token at a time** using all preceding context
2. The process follows the **chain rule of probability**: $P(\mathbf{x}) = \prod_t P(x_t \mid x_{<t})$
3. **Causal left-to-right masking** ensures no future token leakage
4. **Sampling strategies** (temperature, top- p) control output diversity
5. This mechanism underpins every major LLM in production today



Next Steps in Module 1.2

Immediately After This

- Model Sizing and Scaling Laws
- Foundation Models Overview
- Foundation Model Comparison Lab

To Reinforce Learning

- Open the OpenAI or Claude API Playground
- Adjust temperature; observe output diversity
- Trace a multi-turn chat as an autoregressive sequence

Thank You

Autoregressive Generation — LLM Fundamentals
Prof. Sashikumaar Ganesan | IISc Bengaluru | AhamX-Bodhi